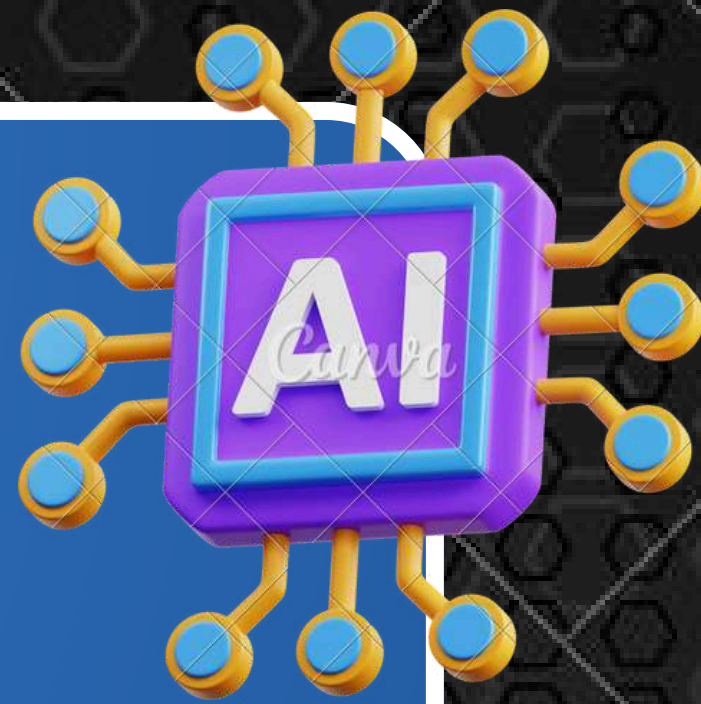


Desarrollo de aplicaciones moviles

Domain-Drive Design

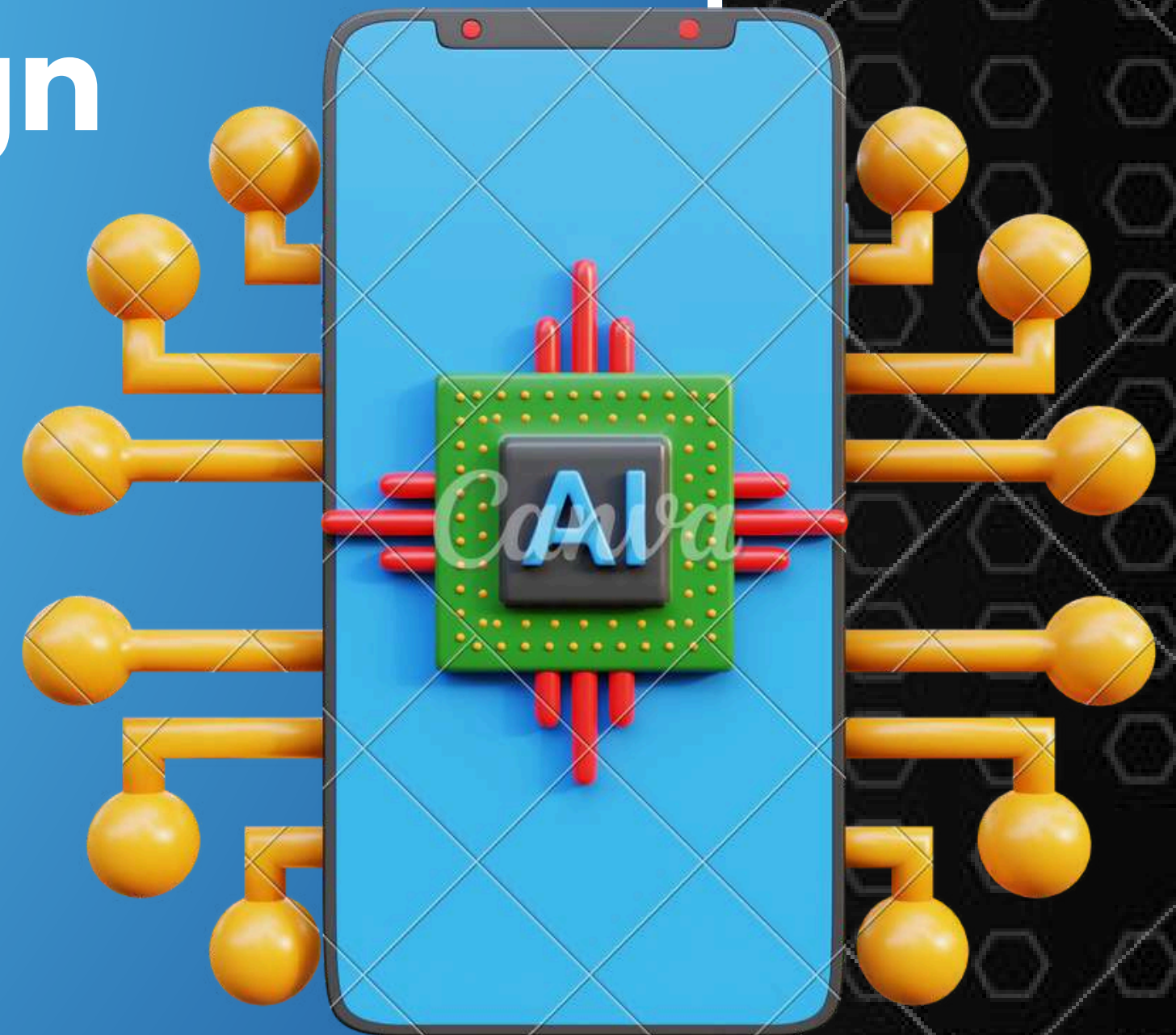
Javier Ramos
Anthony Villarreal
Alexis Chimba
Ariel Reyes



Domain-Driven Design

DDD (Diseño orientado al dominio) es una metodología para desarrollar software centrado en el negocio o dominio principal de una empresa.

Su objetivo es que la lógica del negocio (no la tecnología) sea el eje central del desarrollo.



Como se estructura Domain-Driven Design

1. Dominio

Es el problema real que la empresa quiere resolver.

- Ejemplo: "Gestión de cuentas bancarias", "logística de entregas", "procesamiento de pedidos".

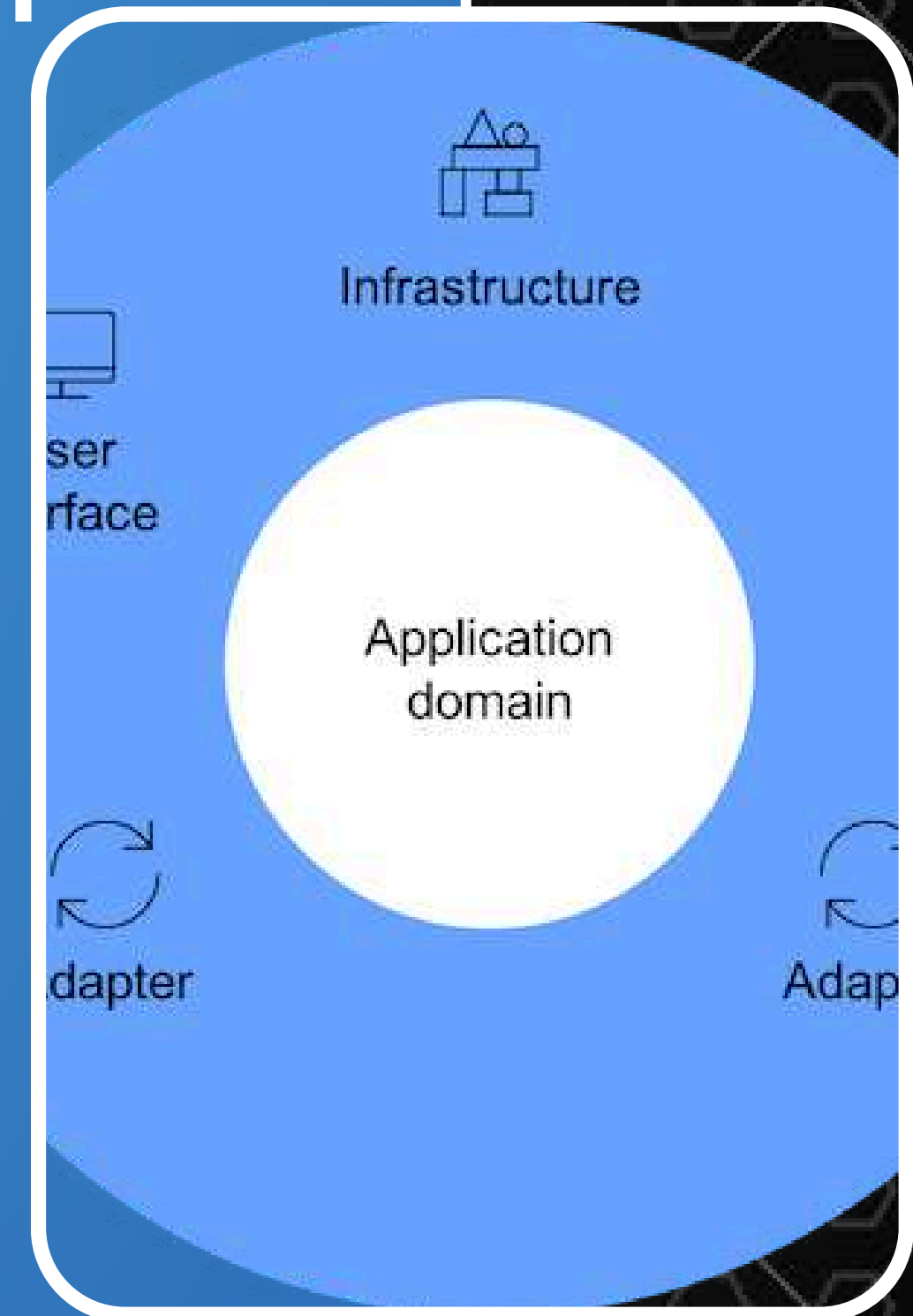
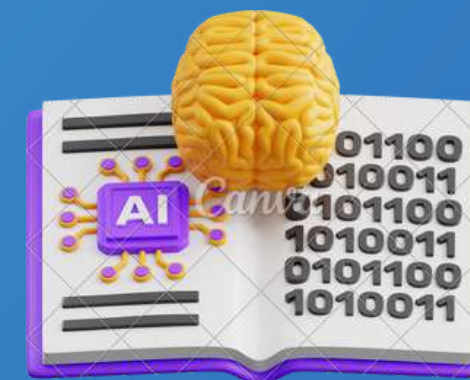


2. Modelo de Dominio

Representación conceptual del dominio en términos de clases, entidades, objetos, relaciones, reglas.

3. Lenguaje Ubicuo

Todos (técnicos y no técnicos) usan los mismos términos exactos para evitar malentendidos.



Como se estructura Domain-Driven Design

4. Entidades y Objetos de valor

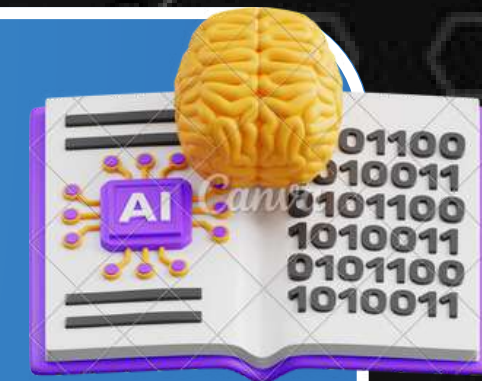
- Entidad: tiene identidad única (por ejemplo: un cliente).
- Objeto de Valor: no tiene identidad, importa su contenido (ej: una dirección).

5. Agregados

- Conjunto de entidades/valores con reglas que aseguran la consistencia de los datos.
- Ejemplo: un pedido y sus líneas de productos.

6. Servicios de Dominio

- Operaciones que no encajan naturalmente en una entidad u objeto de valor, pero pertenecen al dominio.
- Ejemplo: calcular total con descuento.



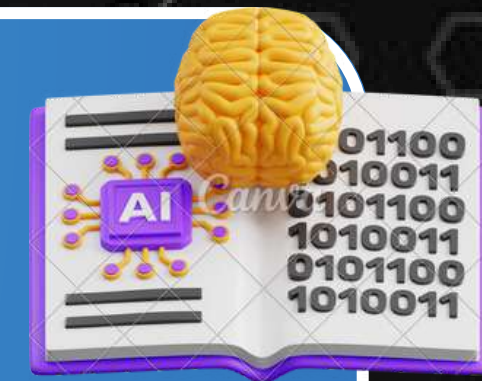
Como se estructura Domain-Driven Design

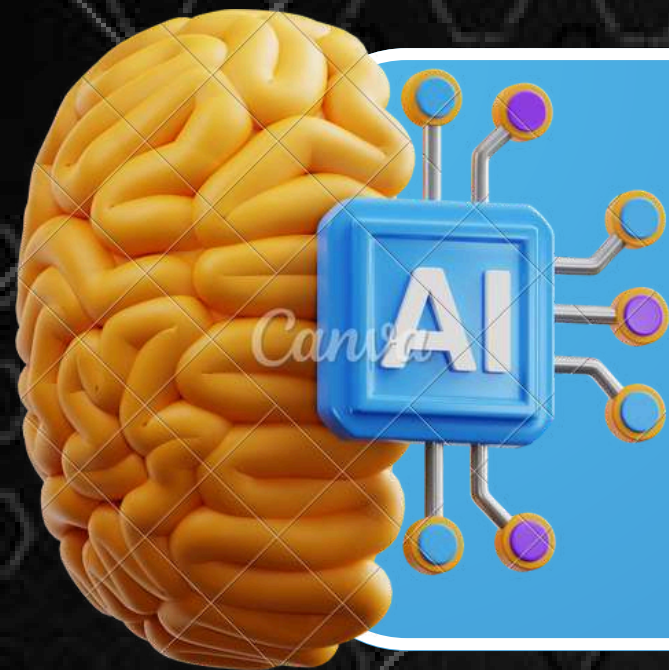
7. Repositorios

- Encargados de recuperar y almacenar agregados del dominio desde la base de datos.

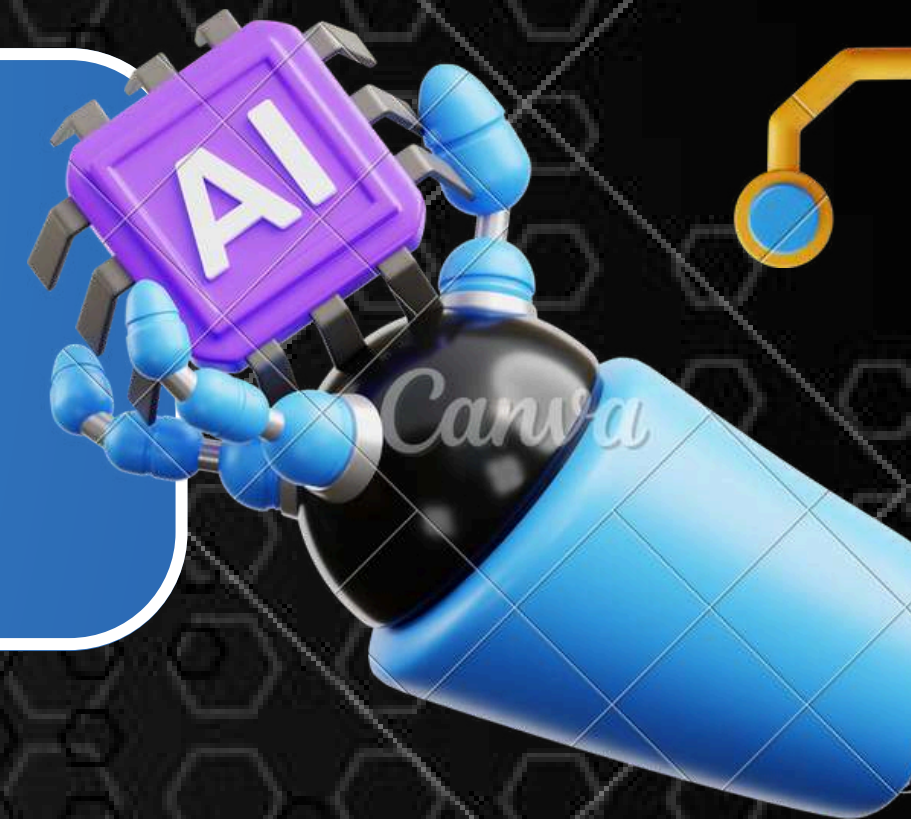
8. Bounded Contexts (Contextos delimitados)

- DDD divide sistemas grandes en módulos separados con lenguaje y reglas propias.
- Ejemplo: "Facturación", "Inventario", "Atención al cliente" son contextos distintos.
-





Aplicación Domain – Driven Design

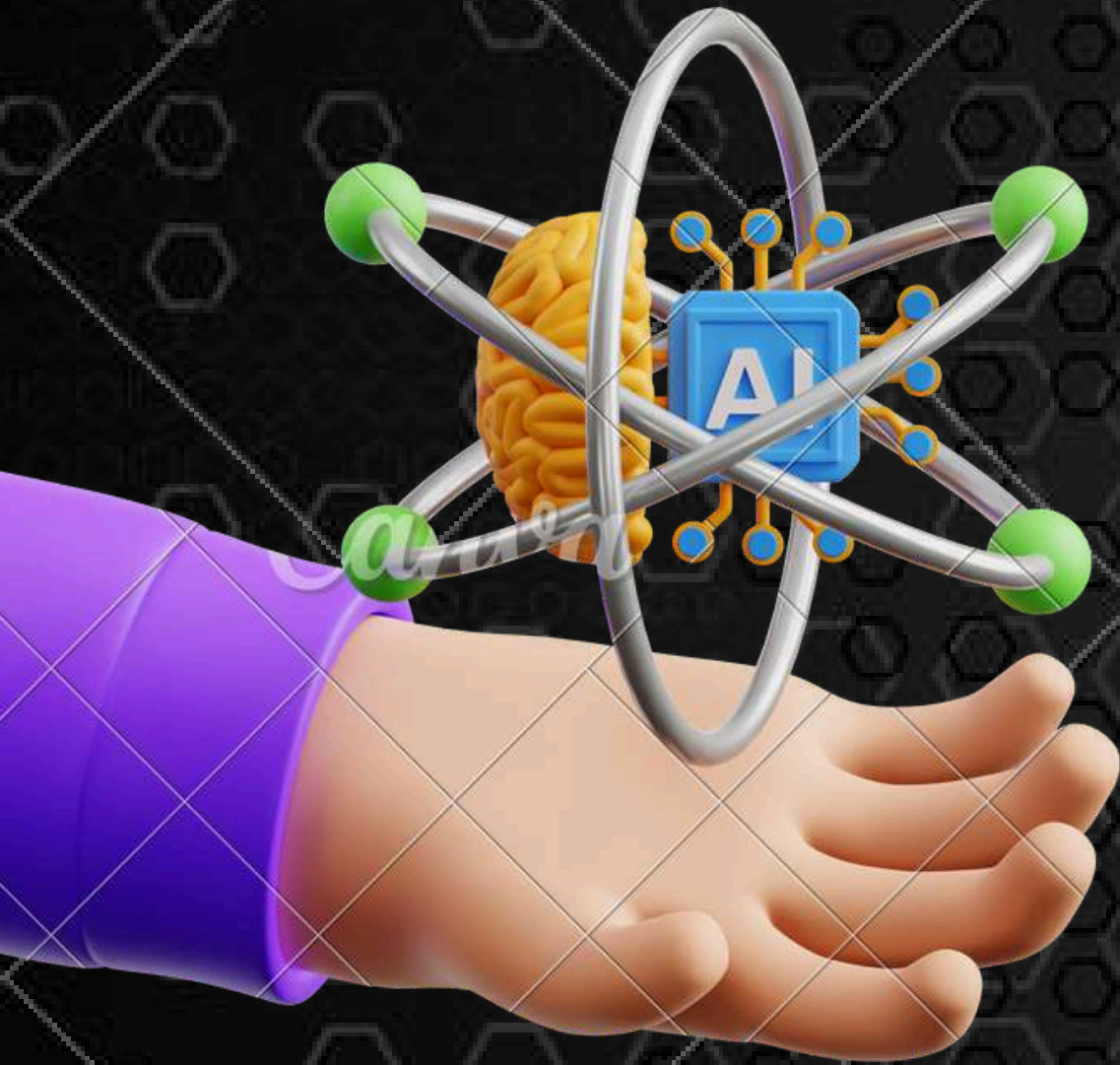


01

Se aplica trabajando junto a expertos del negocio para entender bien el dominio (lo que hace la empresa), creando un modelo claro con entidades, objetos y reglas reales del negocio. Luego, se divide el sistema en módulos (contextos delimitados), se usa un lenguaje común entre técnicos y usuarios, y se refleja todo esto en el código usando servicios, repositorios y eventos que respeten la lógica del negocio.

Bibliografía

- Evans, E. (2004). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley.
- Vernon, V. (2013). Implementing Domain-Driven Design. Addison-Wesley.
- Sommerville, I. (2011). Ingeniería del software (9.ª ed.). Pearson Educación.





Studio
Shodwe

Thank You!

www.reallygreatsite.com

